Lab 12 — Evidencia

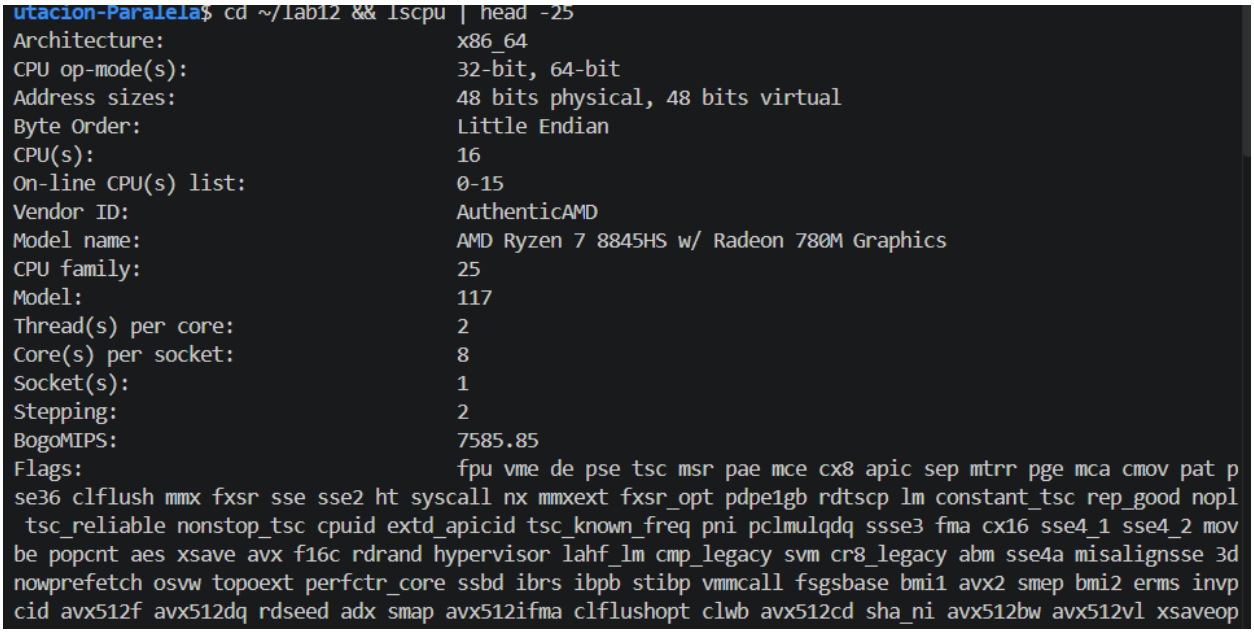
CC3069 Computación Paralela y Distribuida · 10 de octubre de 2026

Cada sección tiene capturas de la terminal de WSL (Ubuntu 24.04.1) y, debajo, el log de texto completo guardado al ejecutar: el comando tal como se corrió, su salida y el código de salida [exit n]. Los archivos originales están en la carpeta evidencia/ del repositorio.

1. CPU

Captura: lscpu

captura: evidencia/capturas/01_lscpu.png — comando repetido el 10/10/2026 18:46, su salida no depende de la corrida



lscpu, lscpu -e y nproc

archivo: evidencia/lscpu.txt

```
$ date
Sat Oct 10 18:22:06 CST 2026
[exit 0]

$ uname -a
Linux Diego 6.6.87.2-microsoft-standard-WSL2 #1 SMP PREEMPT_DYNAMIC Thu Jun  5 18:30:46 UTC 2025 x86_64 x86_64 x86_64
GNU/Linux
[exit 0]

$ lscpu
Architecture:                         x86_64
CPU op-mode(s):                       32-bit, 64-bit
Address sizes:                        48 bits physical, 48 bits virtual
Byte Order:                           Little Endian
CPU(s):                               16
On-line CPU(s) list:                  0-15
```

```
Vendor ID:                           AuthenticAMD
Model name:                          AMD Ryzen 7 8845HS w/ Radeon 780M Graphics
CPU family:                          25
Model:                               117
Thread(s) per core:                  2
Core(s) per socket:                  8
Socket(s):                           1
Stepping:                            2
BogoMIPS:                            7585.85
Flags:                               fpu vme de pse tsc msr pae mce cx8 apic sep mtrr pge mca cmov pat pse36 clflush mmx
fxsr sse sse2 ht syscall nx mmxext fxsr_opt pdpe1gb rdtscp lm constant_tsc rep_good nopl tsc_reliable nonstop_tsc cpuid
extd_apicid tsc_known_freq pni pclmulqdq ssse3 fma cx16 sse4_1 sse4_2 movbe popcnt aes xsave avx f16c rdrand hypervisor
lahf_lm cmp_legacy svm cr8_legacy abm sse4a misalignsse 3dnowprefetch osvw topoext perfctr_core ssbd ibrs ibpb stibp
vmmcall fsgsbase bmi1 avx2 smep bmi2 erms invpcid avx512f avx512dq rdseed adx smap avx512ifma clflushopt clwb avx512cd
sha_ni avx512bw avx512vl xsaveopt xsavec xgetbv1 xsaves avx512_bf16 clzero xsaveerptr arat npt nrip_save tsc_scale
vmcb_clean flushbyasid decodeassists pausefilter pfthreshold v_vmsave_vmload avx512vbmi umip avx512_vbmi2 gfni vaes
vpclmulqdq avx512_vnni avx512_bitalg avx512_vpopcntdq rdpid fsrm
Virtualization:                      AMD-V
Hypervisor vendor:                   Microsoft
Virtualization type:                 full
L1d cache:                           256 KiB (8 instances)
L1i cache:                           256 KiB (8 instances)
L2 cache:                            8 MiB (8 instances)
L3 cache:                            16 MiB (1 instance)
NUMA node(s):                        1
NUMA node0 CPU(s):                   0-15
Vulnerability Gather data sampling:  Not affected
Vulnerability Itlb multihit:         Not affected
Vulnerability L1tf:                  Not affected
Vulnerability Mds:                   Not affected
Vulnerability Meltdown:              Not affected
Vulnerability Mmio stale data:       Not affected
Vulnerability Reg file data sampling: Not affected
Vulnerability Retbleed:              Not affected
Vulnerability Spec rstack overflow:  Vulnerable: Safe RET, no microcode
Vulnerability Spec store bypass:     Mitigation; Speculative Store Bypass disabled via prctl
Vulnerability Spectre v1:            Mitigation; usercopy/swapgs barriers and __user pointer sanitization
Vulnerability Spectre v2:            Mitigation; Retpolines; IBPB conditional; IBRS_FW; STIBP always-on; RSB filling;
PBRSB-eIBRS Not affected; BHI Not affected
Vulnerability Srbds:                 Not affected
Vulnerability Tsx async abort:       Not affected
[exit 0]

$ lscpu -e
CPU NODE SOCKET CORE L1d:L1i:L2:L3 ONLINE
  0    0      0    0 0:0:0:0          yes
  1    0      0    0 0:0:0:0          yes
  2    0      0    1 1:1:1:0          yes
  3    0      0    1 1:1:1:0          yes
  4    0      0    2 2:2:2:0          yes
  5    0      0    2 2:2:2:0          yes
  6    0      0    3 3:3:3:0          yes
  7    0      0    3 3:3:3:0          yes
  8    0      0    4 4:4:4:0          yes
  9    0      0    4 4:4:4:0          yes
 10    0      0    5 5:5:5:0          yes
 11    0      0    5 5:5:5:0          yes
 12    0      0    6 6:6:6:0          yes
 13    0      0    6 6:6:6:0          yes
 14    0      0    7 7:7:7:0          yes
 15    0      0    7 7:7:7:0          yes
[exit 0]

$ nproc
16
[exit 0]
```

Datos de Windows y condiciones de medición

archivo: evidencia/windows_cpu.txt

```
# fecha: 2026-10-10 18:22:19
# Get-CimInstance Win32_Processor
Name                      : AMD Ryzen 7 8845HS w/ Radeon 780M Graphics
NumberOfCores             : 8
NumberOfLogicalProcessors : 16
MaxClockSpeed             : 3801
L2CacheSize               : 8192
L3CacheSize               : 16384

# alimentacion (PowerOnline = conectada)
PowerOnline : True

# procesos relevantes abiertos
Name     Count
----     -----
msedge       9
OneDrive     1

# wsl -l -v
  NAME                STATE           VERSION

* docker-desktop      Stopped         2

  Ubuntu              Running         2

  OracleLinux_8_10    Stopped         2

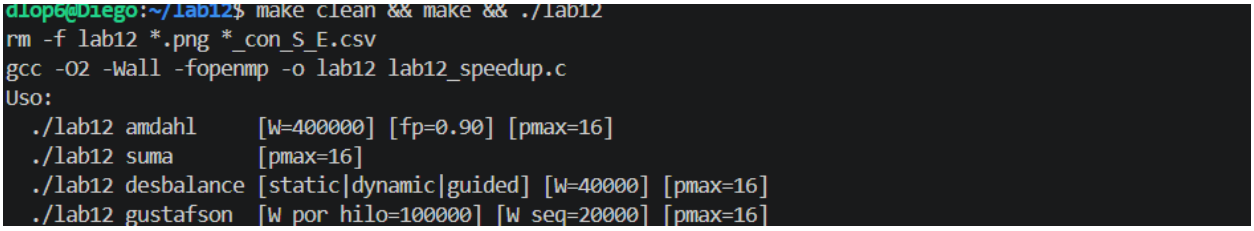


# msedge: procesos de fondo, sin ventanas visibles
# nucleos rendimiento/eficiencia: ni lscpu (WSL) ni Win32_Processor los distinguen; lscpu muestra 8 nucleos con L2 identica
(8 instancias de 1 MiB)
```

2. Compilación (make)

Captura: make clean, make y ./lab12

captura: evidencia/capturas/02_make.png — compilación repetida el 10/10/2026 18:46 con el código entregado (incluye el cambio del reto en tamanos[]); la compilación del código original usado en las mediciones oficiales está en el log make.txt



Integridad del código usado (repo vs ~/lab12)

archivo: evidencia/sha256.txt

```
# repo
ffd0e24ca5172056c7b2e1edea22134f9c88440b4f88b25518475583a1355be1  lab12_speedup.c
ba54ce5f52414286e43d5c9b9ab70d71758b27348140977faa091ab17d055059  Makefile
1168a9b477904f404e7629f876a89b92bd927a240dad8f977a68fbae2fa830a3  graficar.py
# copia en ~/lab12
```

```
ffd0e24ca5172056c7b2e1edea22134f9c88440b4f88b25518475583a1355be1  lab12_speedup.c
ba54ce5f52414286e43d5c9b9ab70d71758b27348140977faa091ab17d055059  Makefile
```

gcc, make y ./lab12 sin argumentos

archivo: evidencia/make.txt

```
$ gcc --version
gcc (Ubuntu 13.3.0-6ubuntu2~24.04.1) 13.3.0
Copyright (C) 2023 Free Software Foundation, Inc.
This is free software; see the source for copying conditions.  There is NO
warranty; not even for MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE.

[exit 0]

$ make
gcc -O2 -Wall -fopenmp -o lab12 lab12_speedup.c
[exit 0]

$ ls -l lab12
-rwxr-xr-x 1 dlop6 dlop6 21760 Oct 10 18:22 lab12
[exit 0]

$ ./lab12
Uso:
  ./lab12 amdahl     [W=400000] [fp=0.90] [pmax=16]
  ./lab12 suma       [pmax=16]
  ./lab12 desbalance [static|dynamic|guided] [W=40000] [pmax=16]
  ./lab12 gustafson  [W_por_hilo=100000] [W_seq=20000] [pmax=16]
[exit 1]
```

matplotlib en WSL

archivo: evidencia/matplotlib.txt

```
$ python3 -c import matplotlib; print('matplotlib', matplotlib.__version__)
matplotlib 3.6.3
[exit 0]
```

3. Predicciones (antes de medir)

Predicciones

archivo: evidencia/predicciones.txt

```
LAB 12 - PREDICCIONES (escritas antes de correr cualquier experimento)
Equipo: AMD Ryzen 7 8845HS, 8 nucleos fisicos / 16 logicos (SMT 2 hilos por nucleo),
L2 1 MiB por nucleo, L3 16 MiB compartida, WSL2 (hipervisor Microsoft). pmax = 16.

PARTE 1 - Amdahl (W = 400000)
Modelo: S(p) = 1 / ((1 - fp) + fp/p)
  fp    S(16) modelo
  1.00  16.00
  0.95   9.14
  0.90   6.40
  0.75   3.37
  0.50   1.88
- fp = 1.00: espero S(16) entre 13 y 15.5 (E 0.81-0.97). Los hilos 9 a 16 son el
  segundo hilo SMT de cada nucleo: como el trabajo es una cadena de operaciones
  dependientes, deja unidades libres y SMT deberia ayudar bastante, pero no como
  un nucleo completo. Ademas el turbo baja con todos los nucleos activos y WSL2
  agrega una capa de virtualizacion.
- fp = 0.90: espero S(16) entre 5.8 y 6.4, un poco debajo del modelo (6.40) por
  overhead de OpenMP y frecuencia.
```

```
PARTE 2 - Suma de N numeros
- Cada parallel for cuesta del orden de 1-2 us (despertar hilos, repartir,
  reduction, barrera) y sumar un elemento cuesta del orden de 1 ns.
- N = 16: la version paralela sera cientos de veces mas lenta.
- Umbral: N* ~ overhead / costo por elemento ~ unos miles de elementos. En la
  rejilla medida (16, 1024, 65536, 1M, 16M) espero S < 1 en N = 1024 y S > 1
  desde N = 65536.
- N grande: con N = 1M (8 MiB, cabe en L3) S cerca de p; con N = 16M (128 MiB,
  no cabe en cache) S lejos de 16 por ancho de banda de memoria.

PARTE 3 - Desbalance (W = 40000, costo(i) = 1 + 2000*i/W, recta creciente)
Trabajo total ~ W*(1 + 1000). Con static el bloque k de p tiene costo promedio
1 + 2000*(k + 0.5)/p (area de un trapecio).
- p = 4, static: ultimo hilo = (1 + 2000*3.5/4) / (4*1001) = 1751/4004 = 0.437
  -> le toca ~43.7 % del trabajo -> S esperado ~ 1/0.437 = 2.29.
- p = 16, static: ultimo hilo ~ 0.121 -> S ~ 8.3 como maximo.
- dynamic y guided: reparten mejor, espero S(16) bastante mayor que static,
  con dynamic algo por debajo del ideal por el costo de pedir bloques de 64.

PARTE 4 - Gustafson (W_seq = 20000, W_por_hilo = 100000)
- Con p = 1 la parte secuencial es 20000/120000 = 1/6 -> fp = 0.83.
- Amdahl con fp = 5/6 (~0.83): limite 1/(1 - 5/6) = 6 (con 0.83 redondeado da 5.9);
  S(16) = 1/(1/6 + (5/6)/16) = 4.57.
- Creo que aqui NO aplica ese limite: el trabajo paralelo crece con p mientras
  la parte secuencial queda fija, asi que t_par casi no cambia y t_seq crece.
  Espero S creciendo casi lineal, del orden de (20000 + 100000*16)/120000 = 13.5
  en p = 16, algo menos por SMT.
```

Hora de inicio y fin de la medición

archivo: evidencia/medicion_inicio.txt

```
# inicio: 2026-10-10 18:24:10
# predicciones.txt modificado: 2026-10-10 18:23:04
ffd0e24ca5172056c7b2e1edea22134f9c88440b4f88b25518475583a1355be1  lab12_speedup.c
# fin: 2026-10-10 18:27:22
```

4. Ejecución de los cuatro modos

Captura: modo amdahl

captura: evidencia/capturas/03_amdahl.png — log registrado durante la medición del 10/10/2026, 18:24–18:27, mostrado en terminal (no es una corrida nueva)



Modo amdahl (5 fracciones)

archivo: evidencia/run_amdahl.txt

```
$ ./lab12 amdahl 400000 1.00 > amdahl_fp100.csv
(checksum 6.799e+12)
[exit 0] [10.5 s]
$ cat amdahl_fp100.csv
modo,N,p,param,t_seq,t_par,t_parte_seq
amdahl,400000,1,fp=1.00,0.495953,0.491456,0.000000
amdahl,400000,2,fp=1.00,0.495953,0.248659,0.000000
amdahl,400000,3,fp=1.00,0.495953,0.165662,0.000000
amdahl,400000,4,fp=1.00,0.495953,0.125658,0.000000
amdahl,400000,5,fp=1.00,0.495953,0.100882,0.000000
amdahl,400000,6,fp=1.00,0.495953,0.083954,0.000000
amdahl,400000,7,fp=1.00,0.495953,0.073210,0.000000
amdahl,400000,8,fp=1.00,0.495953,0.064366,0.000000
amdahl,400000,9,fp=1.00,0.495953,0.057828,0.000000
amdahl,400000,10,fp=1.00,0.495953,0.050729,0.000000
amdahl,400000,11,fp=1.00,0.495953,0.047145,0.000000
amdahl,400000,12,fp=1.00,0.495953,0.044364,0.000000
amdahl,400000,13,fp=1.00,0.495953,0.039930,0.000000
amdahl,400000,14,fp=1.00,0.495953,0.037287,0.000000
amdahl,400000,15,fp=1.00,0.495953,0.035416,0.000000
amdahl,400000,16,fp=1.00,0.495953,0.037563,0.000000

$ ./lab12 amdahl 400000 0.95 > amdahl_fp95.csv
(checksum 6.136e+12)
[exit 0] [12.8 s]
$ cat amdahl_fp95.csv
modo,N,p,param,t_seq,t_par,t_parte_seq
amdahl,400000,1,fp=0.95,0.506102,0.516677,0.026481
amdahl,400000,2,fp=0.95,0.506102,0.264108,0.026522
amdahl,400000,3,fp=0.95,0.506102,0.182152,0.026007
amdahl,400000,4,fp=0.95,0.506102,0.142102,0.025248
amdahl,400000,5,fp=0.95,0.506102,0.120040,0.025366
amdahl,400000,6,fp=0.95,0.506102,0.102988,0.023887
amdahl,400000,7,fp=0.95,0.506102,0.095885,0.025730
amdahl,400000,8,fp=0.95,0.506102,0.087903,0.026346
amdahl,400000,9,fp=0.95,0.506102,0.080095,0.026133
amdahl,400000,10,fp=0.95,0.506102,0.077111,0.026506
amdahl,400000,11,fp=0.95,0.506102,0.073192,0.026626
amdahl,400000,12,fp=0.95,0.506102,0.069351,0.026261
amdahl,400000,13,fp=0.95,0.506102,0.063861,0.026792
amdahl,400000,14,fp=0.95,0.506102,0.062221,0.026687
amdahl,400000,15,fp=0.95,0.506102,0.059837,0.026258
amdahl,400000,16,fp=0.95,0.506102,0.058851,0.027087

$ ./lab12 amdahl 400000 0.90 > amdahl_fp90.csv
(checksum 5.507e+12)
[exit 0] [13.7 s]
$ cat amdahl_fp90.csv
modo,N,p,param,t_seq,t_par,t_parte_seq
amdahl,400000,1,fp=0.90,0.497038,0.489487,0.050959
amdahl,400000,2,fp=0.90,0.497038,0.276077,0.049930
amdahl,400000,3,fp=0.90,0.497038,0.201995,0.051163
amdahl,400000,4,fp=0.90,0.497038,0.164853,0.050832
amdahl,400000,5,fp=0.90,0.497038,0.142319,0.052685
amdahl,400000,6,fp=0.90,0.497038,0.128956,0.052352
amdahl,400000,7,fp=0.90,0.497038,0.117807,0.052764
amdahl,400000,8,fp=0.90,0.497038,0.109802,0.051214
amdahl,400000,9,fp=0.90,0.497038,0.102839,0.051723
amdahl,400000,10,fp=0.90,0.497038,0.098785,0.051766
amdahl,400000,11,fp=0.90,0.497038,0.094445,0.050437
amdahl,400000,12,fp=0.90,0.497038,0.090658,0.052084
amdahl,400000,13,fp=0.90,0.497038,0.088062,0.052549
amdahl,400000,14,fp=0.90,0.497038,0.087358,0.053450
amdahl,400000,15,fp=0.90,0.497038,0.084759,0.052634
```

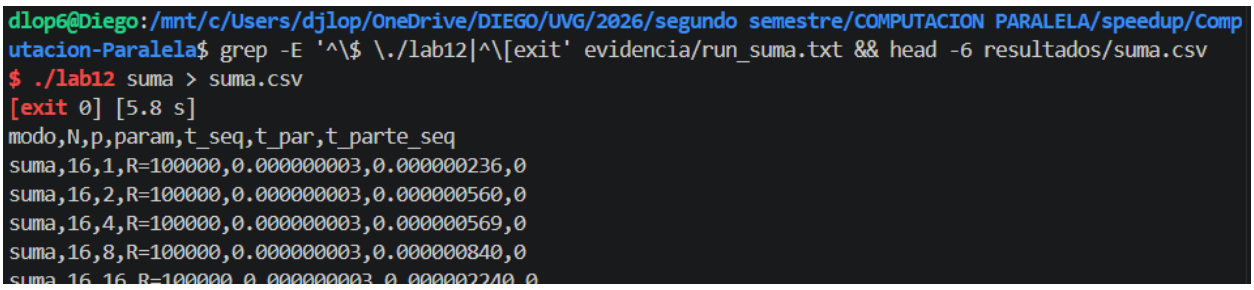
```
amdahl,400000,16,fp=0.90,0.497038,0.088429,0.051955

$ ./lab12 amdahl 400000 0.75 > amdahl_fp75.csv
(checksum 3.825e+12)
[exit 0] [19.3 s]
$ cat amdahl_fp75.csv
modo,N,p,param,t_seq,t_par,t_parte_seq
amdahl,400000,1,fp=0.75,0.497257,0.499997,0.128272
amdahl,400000,2,fp=0.75,0.497257,0.315402,0.128712
amdahl,400000,3,fp=0.75,0.497257,0.252133,0.126297
amdahl,400000,4,fp=0.75,0.497257,0.222364,0.127244
amdahl,400000,5,fp=0.75,0.497257,0.205301,0.127943
amdahl,400000,6,fp=0.75,0.497257,0.191376,0.128905
amdahl,400000,7,fp=0.75,0.497257,0.184607,0.129414
amdahl,400000,8,fp=0.75,0.497257,0.176939,0.129673
amdahl,400000,9,fp=0.75,0.497257,0.171638,0.128525
amdahl,400000,10,fp=0.75,0.497257,0.165467,0.127717
amdahl,400000,11,fp=0.75,0.497257,0.161088,0.126414
amdahl,400000,12,fp=0.75,0.497257,0.161812,0.128242
amdahl,400000,13,fp=0.75,0.497257,0.160314,0.129779
amdahl,400000,14,fp=0.75,0.497257,0.157643,0.129999
amdahl,400000,15,fp=0.75,0.497257,0.153705,0.127283
amdahl,400000,16,fp=0.75,0.497257,0.165760,0.125668

$ ./lab12 amdahl 400000 0.50 > amdahl_fp50.csv
(checksum 1.700e+12)
[exit 0] [26.9 s]
$ cat amdahl_fp50.csv
modo,N,p,param,t_seq,t_par,t_parte_seq
amdahl,400000,1,fp=0.50,0.505505,0.498694,0.254649
amdahl,400000,2,fp=0.50,0.505505,0.388938,0.258900
amdahl,400000,3,fp=0.50,0.505505,0.339046,0.255235
amdahl,400000,4,fp=0.50,0.505505,0.318969,0.255762
amdahl,400000,5,fp=0.50,0.505505,0.304901,0.256314
amdahl,400000,6,fp=0.50,0.505505,0.295999,0.253483
amdahl,400000,7,fp=0.50,0.505505,0.293635,0.255227
amdahl,400000,8,fp=0.50,0.505505,0.286680,0.255848
amdahl,400000,9,fp=0.50,0.505505,0.284382,0.256836
amdahl,400000,10,fp=0.50,0.505505,0.280131,0.254049
amdahl,400000,11,fp=0.50,0.505505,0.281467,0.258271
amdahl,400000,12,fp=0.50,0.505505,0.288059,0.265739
amdahl,400000,13,fp=0.50,0.505505,0.295329,0.273745
amdahl,400000,14,fp=0.50,0.505505,0.279962,0.262193
amdahl,400000,15,fp=0.50,0.505505,0.270500,0.253396
amdahl,400000,16,fp=0.50,0.505505,0.278795,0.257012
```

Captura: modo suma

captura: evidencia/capturas/04_suma.png — log registrado durante la medición del 10/10/2026, 18:24–18:27, mostrado en terminal (no es una corrida nueva)



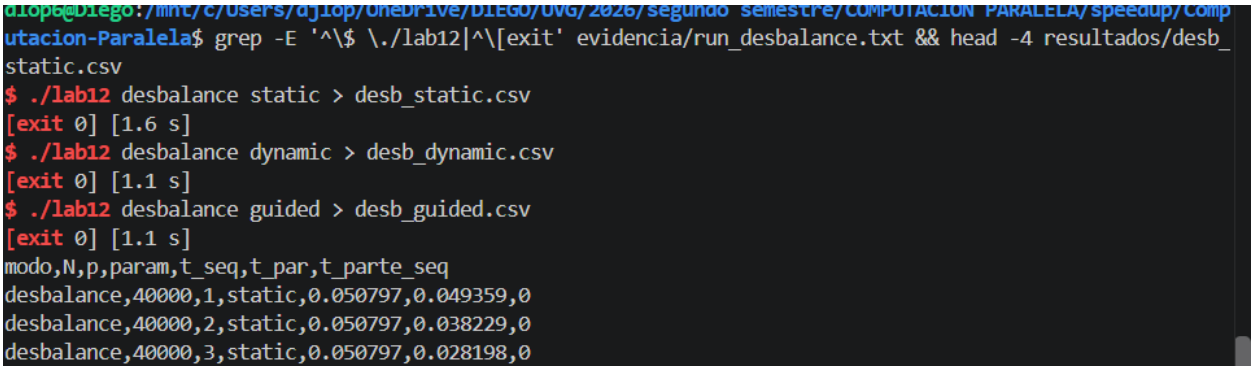
Modo suma

archivo: evidencia/run_suma.txt

```
$ ./lab12 suma > suma.csv
(checksum 2.143e+07)
[exit 0] [5.8 s]
$ cat suma.csv
modo,N,p,param,t_seq,t_par,t_parte_seq
suma,16,1,R=100000,0.000000003,0.000000236,0
suma,16,2,R=100000,0.000000003,0.000000560,0
suma,16,4,R=100000,0.000000003,0.000000569,0
suma,16,8,R=100000,0.000000003,0.000000840,0
suma,16,16,R=100000,0.000000003,0.000002240,0
suma,1024,1,R=48828,0.000000630,0.000000913,0
suma,1024,2,R=48828,0.000000630,0.000000837,0
suma,1024,4,R=48828,0.000000630,0.000000737,0
suma,1024,8,R=48828,0.000000630,0.000000932,0
suma,1024,16,R=48828,0.000000630,0.000002490,0
suma,65536,1,R=762,0.000042861,0.000041283,0
suma,65536,2,R=762,0.000042861,0.000021980,0
suma,65536,4,R=762,0.000042861,0.000011149,0
suma,65536,8,R=762,0.000042861,0.000006651,0
suma,65536,16,R=762,0.000042861,0.000004747,0
suma,1048576,1,R=47,0.000693763,0.000685161,0
suma,1048576,2,R=47,0.000693763,0.000346333,0
suma,1048576,4,R=47,0.000693763,0.000178062,0
suma,1048576,8,R=47,0.000693763,0.000090899,0
suma,1048576,16,R=47,0.000693763,0.000053588,0
suma,16777216,1,R=3,0.011115377,0.011217205,0
suma,16777216,2,R=3,0.011115377,0.007206371,0
suma,16777216,4,R=3,0.011115377,0.003078903,0
suma,16777216,8,R=3,0.011115377,0.002717587,0
suma,16777216,16,R=3,0.011115377,0.003055758,0
```

Captura: modo desbalance

captura: evidencia/capturas/05_desbalance.png — log registrado durante la medición del 10/10/2026, 18:24–18:27, mostrado en terminal (no es una corrida nueva)



Modo desbalance (static, dynamic, guided)

archivo: evidencia/run_desbalance.txt

```
$ ./lab12 desbalance static > desb_static.csv
(checksum 6.799e+10)
[exit 0] [1.6 s]
$ cat desb_static.csv
modo,N,p,param,t_seq,t_par,t_parte_seq
desbalance,40000,1,static,0.050797,0.049359,0
desbalance,40000,2,static,0.050797,0.038229,0
desbalance,40000,3,static,0.050797,0.028198,0
desbalance,40000,4,static,0.050797,0.021926,0
desbalance,40000,5,static,0.050797,0.017888,0
desbalance,40000,6,static,0.050797,0.014848,0
desbalance,40000,7,static,0.050797,0.013700,0
```

```
desbalance,40000,8,static,0.050797,0.012136,0
desbalance,40000,9,static,0.050797,0.013892,0
desbalance,40000,10,static,0.050797,0.010031,0
desbalance,40000,11,static,0.050797,0.009444,0
desbalance,40000,12,static,0.050797,0.008684,0
desbalance,40000,13,static,0.050797,0.007794,0
desbalance,40000,14,static,0.050797,0.007256,0
desbalance,40000,15,static,0.050797,0.006837,0
desbalance,40000,16,static,0.050797,0.006692,0

$ ./lab12 desbalance dynamic > desb_dynamic.csv
(checksum 6.799e+10)
[exit 0] [1.1 s]
$ cat desb_dynamic.csv
modo,N,p,param,t_seq,t_par,t_parte_seq
desbalance,40000,1,dynamic,0.049594,0.049332,0
desbalance,40000,2,dynamic,0.049594,0.024677,0
desbalance,40000,3,dynamic,0.049594,0.016605,0
desbalance,40000,4,dynamic,0.049594,0.012474,0
desbalance,40000,5,dynamic,0.049594,0.009850,0
desbalance,40000,6,dynamic,0.049594,0.008226,0
desbalance,40000,7,dynamic,0.049594,0.007017,0
desbalance,40000,8,dynamic,0.049594,0.006193,0
desbalance,40000,9,dynamic,0.049594,0.005818,0
desbalance,40000,10,dynamic,0.049594,0.005156,0
desbalance,40000,11,dynamic,0.049594,0.004728,0
desbalance,40000,12,dynamic,0.049594,0.004305,0
desbalance,40000,13,dynamic,0.049594,0.003973,0
desbalance,40000,14,dynamic,0.049594,0.003791,0
desbalance,40000,15,dynamic,0.049594,0.003552,0
desbalance,40000,16,dynamic,0.049594,0.003319,0

$ ./lab12 desbalance guided > desb_guided.csv
(checksum 6.799e+10)
[exit 0] [1.1 s]
$ cat desb_guided.csv
modo,N,p,param,t_seq,t_par,t_parte_seq
desbalance,40000,1,guided,0.050294,0.048940,0
desbalance,40000,2,guided,0.050294,0.024738,0
desbalance,40000,3,guided,0.050294,0.016261,0
desbalance,40000,4,guided,0.050294,0.012294,0
desbalance,40000,5,guided,0.050294,0.009992,0
desbalance,40000,6,guided,0.050294,0.008249,0
desbalance,40000,7,guided,0.050294,0.007131,0
desbalance,40000,8,guided,0.050294,0.006367,0
desbalance,40000,9,guided,0.050294,0.005628,0
desbalance,40000,10,guided,0.050294,0.005069,0
desbalance,40000,11,guided,0.050294,0.004659,0
desbalance,40000,12,guided,0.050294,0.004278,0
desbalance,40000,13,guided,0.050294,0.003922,0
desbalance,40000,14,guided,0.050294,0.003825,0
desbalance,40000,15,guided,0.050294,0.003532,0
desbalance,40000,16,guided,0.050294,0.003313,0
```

Captura: modo gustafson

captura: evidencia/capturas/06_gustafson.png — log registrado durante la medición del 10/10/2026, 18:24–18:27, mostrado en terminal (no es una corrida nueva)

```
utacion-Paralela$ grep -E '^\$ \./lab12|^\[exit' evidencia/run_gustafson.txt && head -6 resultados/gusta
son.csv
$ ./lab12 gustafson > gustafson.csv
[exit 0] [99.1 s]
modo,N,p,param,t_seq,t_par,t_parte_seq
gustafson,120000,1,Wseq=20000,0.152328,0.148848,0.025392
gustafson,220000,2,Wseq=20000,0.272636,0.152012,0.025548
gustafson,320000,3,Wseq=20000,0.398348,0.149760,0.025326
gustafson,420000,4,Wseq=20000,0.517182,0.155956,0.026527
gustafson,520000,5,Wseq=20000,0.652766,0.152881,0.025770
dlop6@Diego:/mnt/c/Users/dilop/OneDrive/DIEGO/UVG/2026/segundo semestre/COMPUTACION PARALELA/speedup/Com
```

Modo gustafson

archivo: evidencia/run_gustafson.txt

```
$ ./lab12 gustafson > gustafson.csv
(checksum 7.479e+13)
[exit 0] [99.1 s]
$ cat gustafson.csv
modo,N,p,param,t_seq,t_par,t_parte_seq
gustafson,120000,1,Wseq=20000,0.152328,0.148848,0.025392
gustafson,220000,2,Wseq=20000,0.272636,0.152012,0.025548
gustafson,320000,3,Wseq=20000,0.398348,0.149760,0.025326
gustafson,420000,4,Wseq=20000,0.517182,0.155956,0.026527
gustafson,520000,5,Wseq=20000,0.652766,0.152881,0.025770
gustafson,620000,6,Wseq=20000,0.761233,0.158443,0.026710
gustafson,720000,7,Wseq=20000,0.899427,0.152600,0.025698
gustafson,820000,8,Wseq=20000,1.032283,0.153228,0.027133
gustafson,920000,9,Wseq=20000,1.130357,0.160921,0.026626
gustafson,1020000,10,Wseq=20000,1.287582,0.159949,0.026760
gustafson,1120000,11,Wseq=20000,1.425094,0.160359,0.026558
gustafson,1220000,12,Wseq=20000,1.562639,0.159755,0.026941
gustafson,1320000,13,Wseq=20000,1.685479,0.162104,0.028531
gustafson,1420000,14,Wseq=20000,1.802958,0.176959,0.027089
gustafson,1520000,15,Wseq=20000,1.938491,0.169202,0.027596
gustafson,1620000,16,Wseq=20000,2.063817,0.230183,0.026977
```

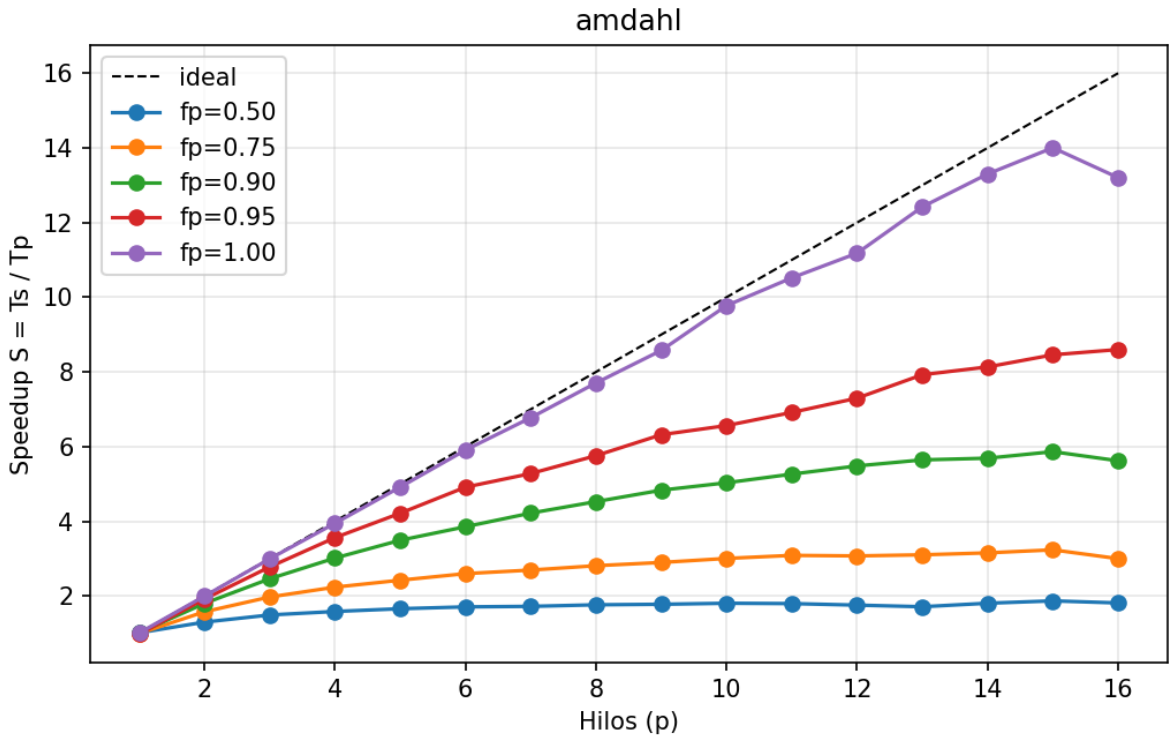
5. Gráficas

graficar.py sobre los 10 CSV originales

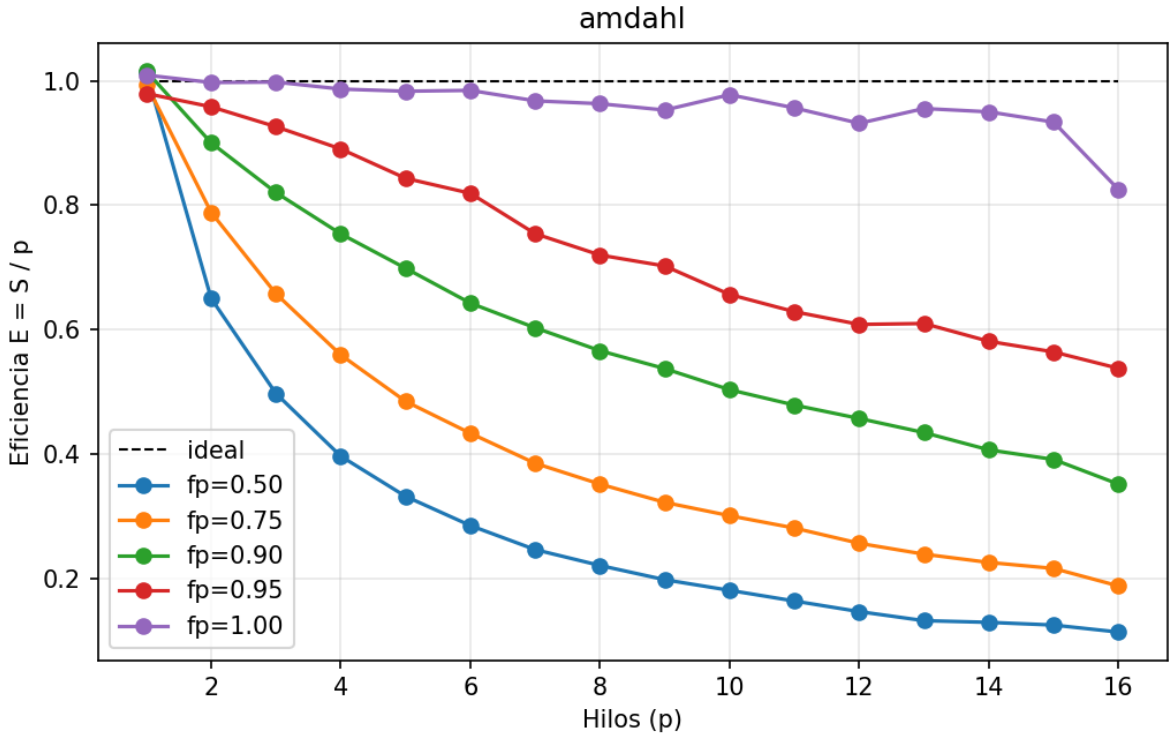
archivo: evidencia/run_graficar.txt

```
$ python3 ../graficar.py amdahl_fp100.csv amdahl_fp95.csv amdahl_fp90.csv amdahl_fp75.csv amdahl_fp50.csv suma.csv
desb_static.csv desb_dynamic.csv desb_guided.csv gustafson.csv
[amdahl] 80 filas
  -> amdahl_speedup.png
  -> amdahl_eficiencia.png
[suma] 25 filas
  -> suma_speedup.png
[desbalance] 48 filas
  -> desbalance_speedup.png
  -> desbalance_eficiencia.png
[gustafson] 16 filas
  -> gustafson_speedup.png
  -> gustafson_eficiencia.png
[exit 0]
```

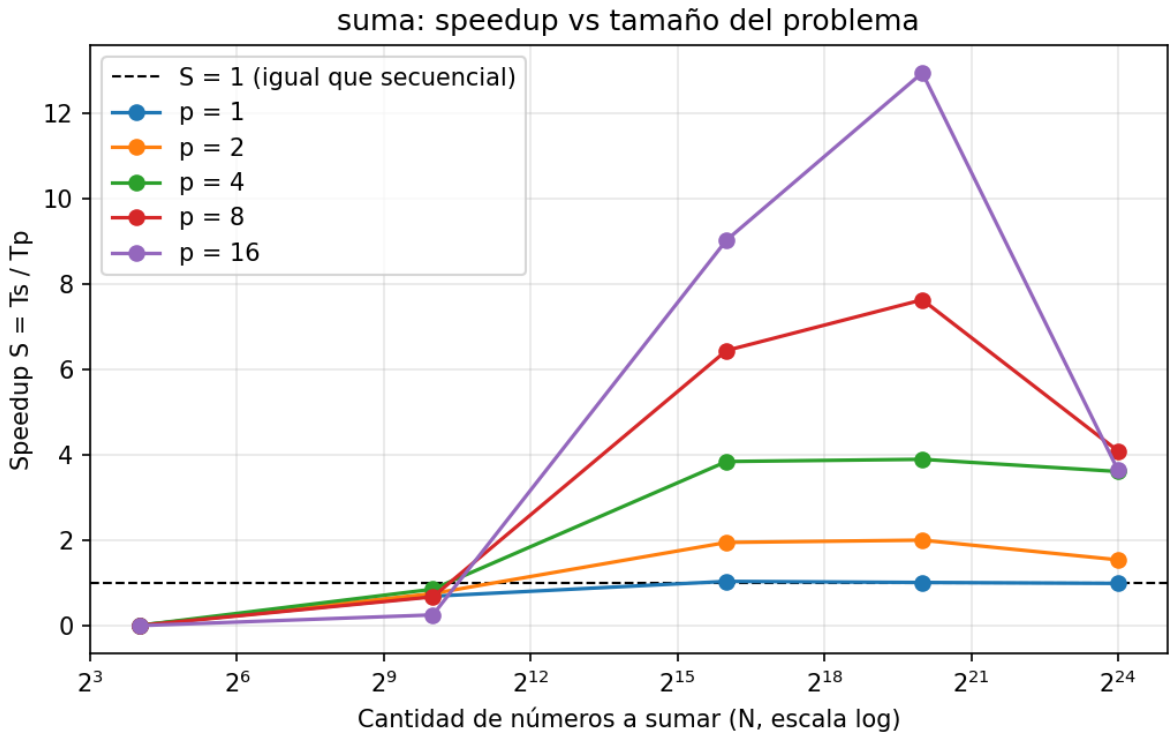
amdahl_speedup.png



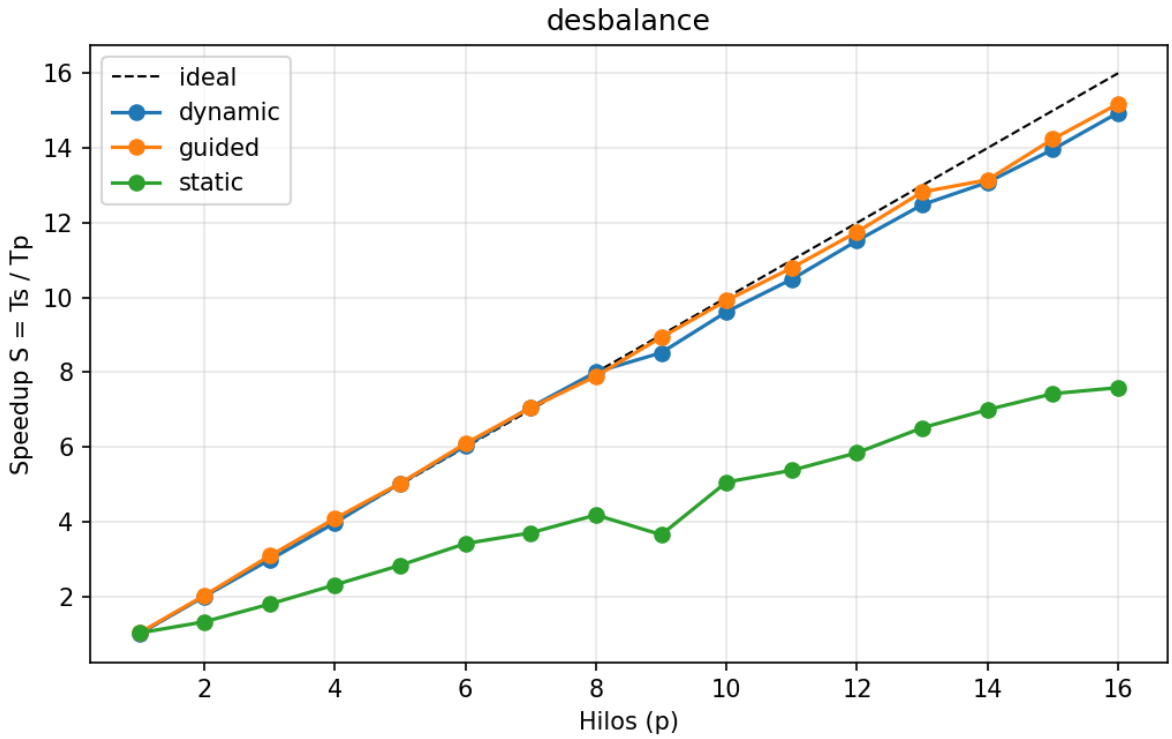
amdahl_eficiencia.png



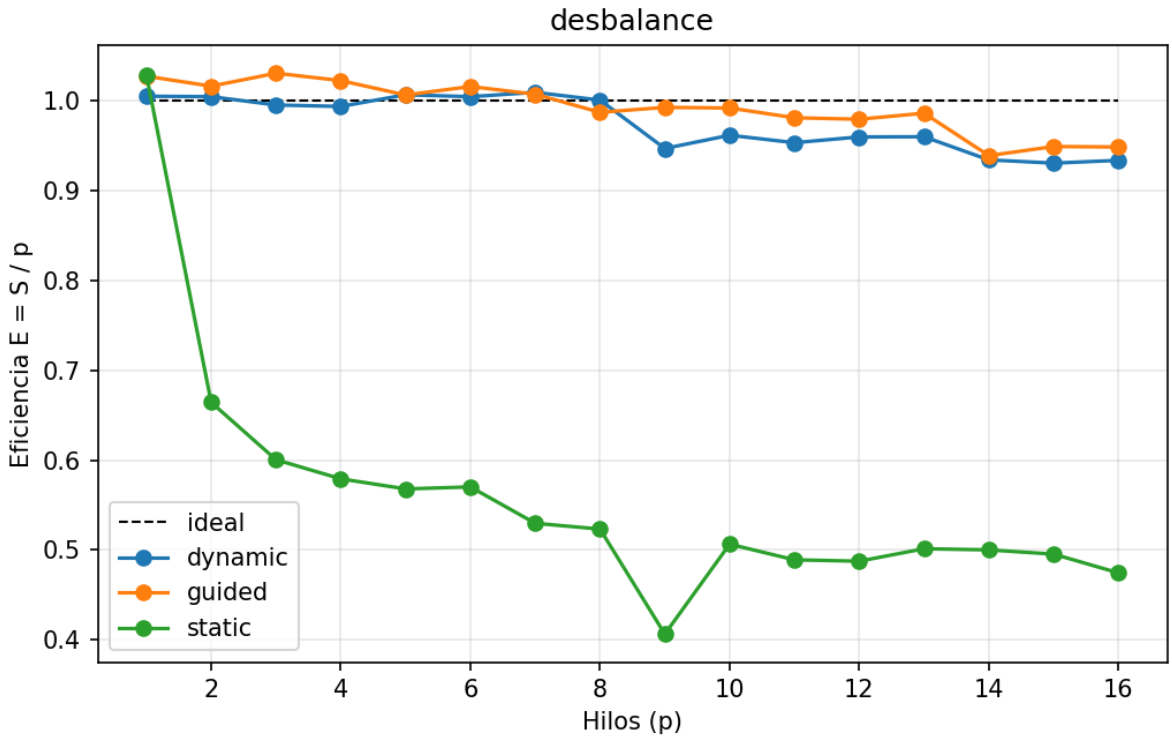
suma_speedup.png



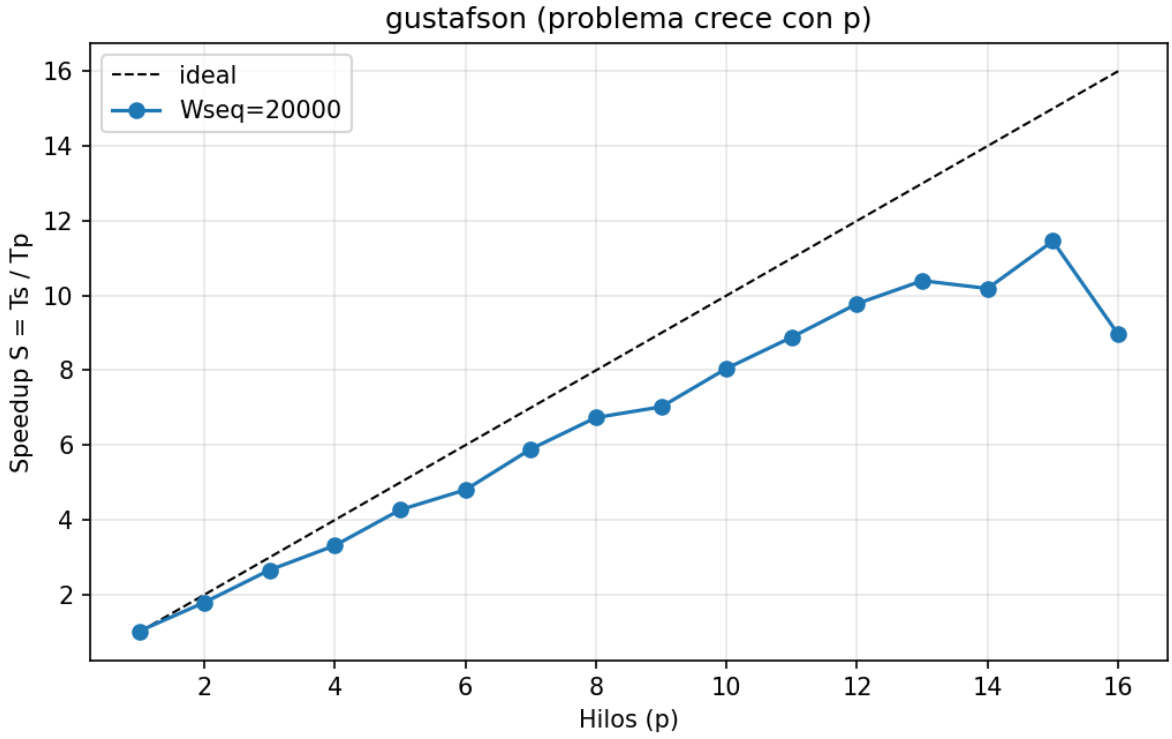
desbalance_speedup.png



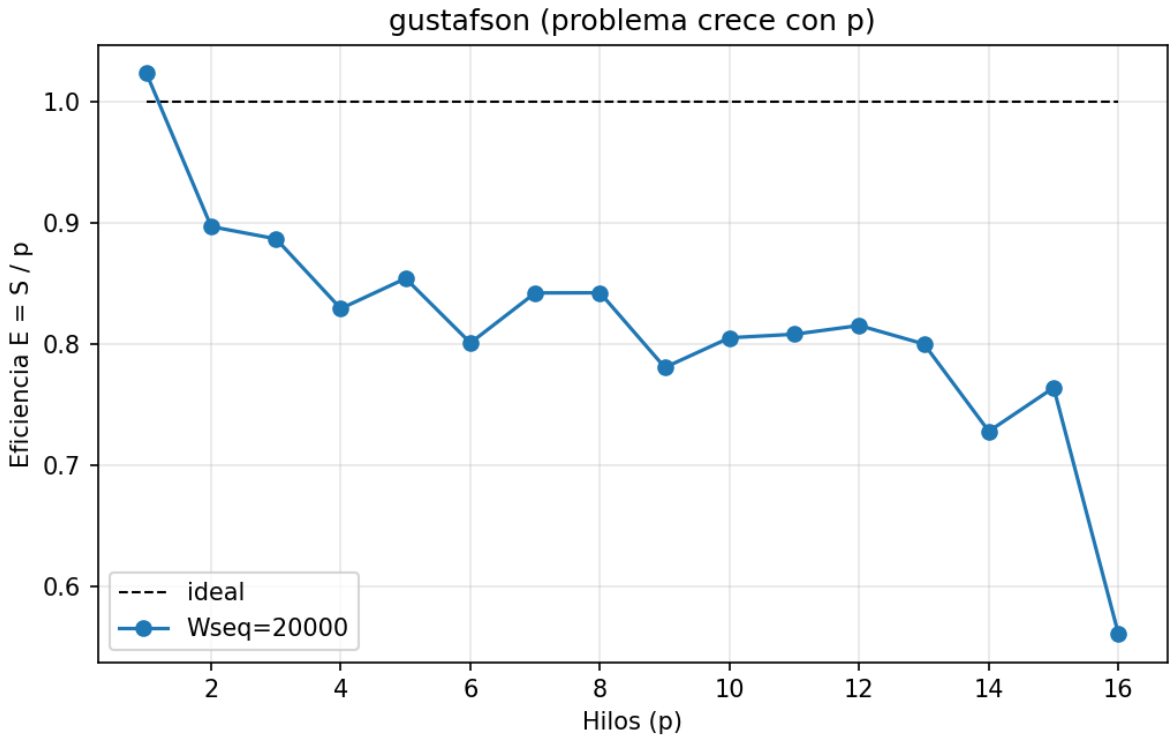
desbalance_eficiencia.png



gustafson_speedup.png



gustafson_eficiencia.png



6. Análisis

analizar_partes12.py

archivo: evidencia/analisis_partes12.txt

```
$ python3 analizar_partes12.py resultados
## Tabla oficial Parte 1

| fp | S(p=2) | S(p=4) | S(pmax) | E(pmax) | p con F máxima |
|---|---|---|---|---|---|
| 1.00 | 1.995 | 3.947 | 13.203 (p=16) | 0.825 | 15 |
| 0.95 | 1.916 | 3.562 | 8.600 (p=16) | 0.537 | 13 |
| 0.90 | 1.800 | 3.015 | 5.621 (p=16) | 0.351 | 9 |
| 0.75 | 1.577 | 2.236 | 3.000 (p=16) | 0.187 | 3 |
| 0.50 | 1.300 | 1.585 | 1.813 (p=16) | 0.113 | 1 |

## Cálculos a mano (fp = 0.90)

p = 2:  t_seq = 0.497038  t_par = 0.276077
  S = Ts/Tp = 0.497038/0.276077 = 1.800
  E = S/p   = 1.800/2 = 0.900
  C = p·Tp  = 2·0.276077 = 0.552154 hilo-s
  F = S/C   = 1.800/0.552154 = 3.261

p = 16:  t_seq = 0.497038  t_par = 0.088429
  S = Ts/Tp = 0.497038/0.088429 = 5.621
  E = S/p   = 5.621/16 = 0.351
  C = p·Tp  = 16·0.088429 = 1.414864 hilo-s
  F = S/C   = 5.621/1.414864 = 3.973

## P1.4 y P1.6 (fp = 0.90)
  S predicho = 1/(0.10+0.90/16) = 6.400;  S medido = 5.621;  error = -12.2%
  fp despejado = (p/(p-1))·(1-1/S) = 0.8769  (nominal 0.90)
```

```
  P1.5: límite 1/(1-fp) = 10.0;  S(1000) = 9.911

## P1.2 (fp = 0.75): ganancia marginal ΔS = S(p) - S(p-1)
  hilo 2: ΔS = +0.582
  hilo 3: ΔS = +0.396
  hilo 4: ΔS = +0.264
  hilo 8: ΔS = +0.117
  hilo 15: ΔS = +0.081
  hilo 16: ΔS = -0.235

## P2.1 / P2.2 (amdahl_fp100, p = 1)
  t_seq = 0.495953  t_par(p=1) = 0.491456  -> OpenMP 1 hilo es -0.91% respecto al secuencial
  S(pmax) honesto = 13.203;  con Ts := Tp(1) = 13.084

## Parte 2b (suma)
  P2.3: N=16, p=16: S = 0.0013 -> la versión paralela es 746.7x más lenta (t_seq=3.000e-09s, t_par=2.240e-06s)
  P2.6: menor N con S > 1 por p:
    p=1: 65536
    p=2: 65536
    p=4: 65536
    p=8: 65536
    p=16: 65536
  Estimación: O ≈ t_par(N=16) = 2.240e-06s, c = 6.540e-10s/elemento -> N* ≈ O/(c(1-1/p)) = 3653
  P2.7: N=16777216, p=16: S = 3.638 (ideal 16), E = 0.227

  Tabla S por N y p:
  N | p=1 | p=2 | p=4 | p=8 | p=16
  16 | 0.013 | 0.005 | 0.005 | 0.004 | 0.001
  1024 | 0.690 | 0.753 | 0.855 | 0.676 | 0.253
  65536 | 1.038 | 1.950 | 3.844 | 6.444 | 9.029
  1048576 | 1.013 | 2.003 | 3.896 | 7.632 | 12.946
  16777216 | 0.991 | 1.542 | 3.610 | 4.090 | 3.638
[exit 0]
```

analizar_partes34.py

archivo: evidencia/analisis_partes34.txt

```
$ python3 analizar_partes34.py resultados
## P3.1: S(pmax) por schedule

  static   p=16: Tp = 0.006692  S = 7.591  E = 0.474
  dynamic  p=16: Tp = 0.003319  S = 14.942  E = 0.934
  guided   p=16: Tp = 0.003313  S = 15.181  E = 0.949
  dynamic vs static: ΔS = +7.352  (+96.9%)
  guided vs static: ΔS = +7.590  (+100.0%)
  guided vs dynamic: ΔS = +0.238  (+1.6%)

## P3.2: hilo más cargado con static (W = 40000)

  p=2: fracción trapecio = 0.7498  exacta = 0.7499  -> S predicho = 1/f = 1.334  S medido = 1.329  error = -0.4%
  p=4: fracción trapecio = 0.4373  exacta = 0.4374  -> S predicho = 1/f = 2.286  S medido = 2.317  error = +1.3%
  p=16: fracción trapecio = 0.1210  exacta = 0.1211  -> S predicho = 1/f = 8.260  S medido = 7.591  error = -8.1%

## P3.3: espera de los hilos con static (E predicho = 1/(p·f))

  p=2: E predicho = 0.667  E medido = 0.664  -> los hilos esperan ~34% del tiempo
  p=4: E predicho = 0.572  E medido = 0.579  -> los hilos esperan ~42% del tiempo
  p=8: E predicho = 0.533  E medido = 0.523  -> los hilos esperan ~48% del tiempo
  p=16: E predicho = 0.516  E medido = 0.474  -> los hilos esperan ~53% del tiempo

  E static por p: p1=1.029  p2=0.664  p3=0.600  p4=0.579  p5=0.568  p6=0.570  p7=0.530  p8=0.523  p9=0.406  p10=0.506
p11=0.489  p12=0.487  p13=0.501  p14=0.500  p15=0.495  p16=0.474

## P3.4: costo de repartir en tiempo de ejecución (p = 1)

  static  : t_seq = 0.050797  t_par(1) = 0.049359  (-2.83%)
  dynamic : t_seq = 0.049594  t_par(1) = 0.049332  (-0.53%)
```

```
  guided  : t_seq = 0.050294  t_par(1) = 0.048940  (-2.69%)
  dynamic reparte 625 bloques de 64 iteraciones (W/64)

## P4.1: evolución de t_seq y t_par

  t_seq: 0.152328 -> 2.063817  (x13.55)
  t_par: 0.148848 -> 0.230183  (x1.55)

## P4.2: Amdahl con la fp de p = 1

  fp = 1 - 20000/120000 = 0.8333;  límite 1/(1-fp) = 6.00;  Amdahl S(16) = 4.571;  medido S(16) = 8.966

## P4.3 y P4.4: s = t_parte_seq / t_par y modelo S = s + (1-s)·p

  p | t_seq | t_par | t_parte_seq | s | S medido | S modelo | error
  1 | 0.152328 | 0.148848 | 0.025392 | 0.1706 | 1.023 | 1.000 | +2.3%
  2 | 0.272636 | 0.152012 | 0.025548 | 0.1681 | 1.794 | 1.832 | -2.1%
  3 | 0.398348 | 0.149760 | 0.025326 | 0.1691 | 2.660 | 2.662 | -0.1%
  4 | 0.517182 | 0.155956 | 0.026527 | 0.1701 | 3.316 | 3.490 | -5.0%
  5 | 0.652766 | 0.152881 | 0.025770 | 0.1686 | 4.270 | 4.326 | -1.3%
  6 | 0.761233 | 0.158443 | 0.026710 | 0.1686 | 4.804 | 5.157 | -6.8%
  7 | 0.899427 | 0.152600 | 0.025698 | 0.1684 | 5.894 | 5.990 | -1.6%
  8 | 1.032283 | 0.153228 | 0.027133 | 0.1771 | 6.737 | 6.760 | -0.3%
  9 | 1.130357 | 0.160921 | 0.026626 | 0.1655 | 7.024 | 7.676 | -8.5%
  10 | 1.287582 | 0.159949 | 0.026760 | 0.1673 | 8.050 | 8.494 | -5.2%
  11 | 1.425094 | 0.160359 | 0.026558 | 0.1656 | 8.887 | 9.344 | -4.9%
  12 | 1.562639 | 0.159755 | 0.026941 | 0.1686 | 9.781 | 10.145 | -3.6%
  13 | 1.685479 | 0.162104 | 0.028531 | 0.1760 | 10.398 | 10.888 | -4.5%
  14 | 1.802958 | 0.176959 | 0.027089 | 0.1531 | 10.189 | 12.010 | -15.2%
  15 | 1.938491 | 0.169202 | 0.027596 | 0.1631 | 11.457 | 12.717 | -9.9%
  16 | 2.063817 | 0.230183 | 0.026977 | 0.1172 | 8.966 | 14.242 | -37.0%

  s: mín = 0.1172  máx = 0.1771  promedio = 0.1648
[exit 0]
```

7. Reto opcional

Cambio en tamanos[] y recompilación

archivo: evidencia/reto_make.txt

```
# inicio: 2026-10-10 18:30:07
$ grep -n tamanos\[\] = lab12_speedup.c
114:    long tamanos[] = {65536, 131072, 262144, 524288, 1048576, 2097152, 4194304};
[exit 0]

$ sha256sum lab12_speedup.c
50b7d98e790f17342488ba05cf4a8a0599ab477dfbfb78a668187251096cf92e  lab12_speedup.c
[exit 0]

$ make
gcc -O2 -Wall -fopenmp -o lab12 lab12_speedup.c
[exit 0]
```

5 corridas independientes de ./lab12 suma

archivo: evidencia/run_reto.txt

```
$ ./lab12 suma > suma_reto_run1.csv
[exit 0]
(checksum 5.576e+05)
$ cat suma_reto_run1.csv
modo,N,p,param,t_seq,t_par,t_parte_seq
suma,65536,1,R=762,0.000042736,0.000043367,0
suma,65536,2,R=762,0.000042736,0.000022624,0
```

```
suma,65536,4,R=762,0.000042736,0.000012067,0
suma,65536,8,R=762,0.000042736,0.000008115,0
suma,65536,16,R=762,0.000042736,0.000006088,0
suma,131072,1,R=381,0.000087203,0.000086938,0
suma,131072,2,R=381,0.000087203,0.000044259,0
suma,131072,4,R=381,0.000087203,0.000022835,0
suma,131072,8,R=381,0.000087203,0.000012048,0
suma,131072,16,R=381,0.000087203,0.000007734,0
suma,262144,1,R=190,0.000172203,0.000178394,0
suma,262144,2,R=190,0.000172203,0.000087922,0
suma,262144,4,R=190,0.000172203,0.000045607,0
suma,262144,8,R=190,0.000172203,0.000024739,0
suma,262144,16,R=190,0.000172203,0.000016101,0
suma,524288,1,R=95,0.000343536,0.000357211,0
suma,524288,2,R=95,0.000343536,0.000177957,0
suma,524288,4,R=95,0.000343536,0.000089457,0
suma,524288,8,R=95,0.000343536,0.000049091,0
suma,524288,16,R=95,0.000343536,0.000148124,0
suma,1048576,1,R=47,0.000700092,0.000671879,0
suma,1048576,2,R=47,0.000700092,0.000354067,0
suma,1048576,4,R=47,0.000700092,0.000189776,0
suma,1048576,8,R=47,0.000700092,0.000092864,0
suma,1048576,16,R=47,0.000700092,0.000049903,0
suma,2097152,1,R=23,0.001376140,0.001400252,0
suma,2097152,2,R=23,0.001376140,0.000697042,0
suma,2097152,4,R=23,0.001376140,0.000350308,0
suma,2097152,8,R=23,0.001376140,0.000197986,0
suma,2097152,16,R=23,0.001376140,0.000101536,0
suma,4194304,1,R=11,0.002727044,0.002747277,0
suma,4194304,2,R=11,0.002727044,0.001578563,0
suma,4194304,4,R=11,0.002727044,0.000748576,0
suma,4194304,8,R=11,0.002727044,0.000437871,0
suma,4194304,16,R=11,0.002727044,0.000472010,0

$ ./lab12 suma > suma_reto_run2.csv
[exit 0]
(checksum 5.576e+05)
$ cat suma_reto_run2.csv
modo,N,p,param,t_seq,t_par,t_parte_seq
suma,65536,1,R=762,0.000041958,0.000042404,0
suma,65536,2,R=762,0.000041958,0.000022218,0
suma,65536,4,R=762,0.000041958,0.000011579,0
suma,65536,8,R=762,0.000041958,0.000006505,0
suma,65536,16,R=762,0.000041958,0.000005148,0
suma,131072,1,R=381,0.000084452,0.000084513,0
suma,131072,2,R=381,0.000084452,0.000042159,0
suma,131072,4,R=381,0.000084452,0.000022908,0
suma,131072,8,R=381,0.000084452,0.000013455,0
suma,131072,16,R=381,0.000084452,0.000007897,0
suma,262144,1,R=190,0.000170888,0.000170789,0
suma,262144,2,R=190,0.000170888,0.000093521,0
suma,262144,4,R=190,0.000170888,0.000047744,0
suma,262144,8,R=190,0.000170888,0.000023616,0
suma,262144,16,R=190,0.000170888,0.000014414,0
suma,524288,1,R=95,0.000339023,0.000338796,0
suma,524288,2,R=95,0.000339023,0.000184195,0
suma,524288,4,R=95,0.000339023,0.000088559,0
suma,524288,8,R=95,0.000339023,0.000046131,0
suma,524288,16,R=95,0.000339023,0.000033702,0
suma,1048576,1,R=47,0.000690674,0.000676410,0
suma,1048576,2,R=47,0.000690674,0.000338691,0
suma,1048576,4,R=47,0.000690674,0.000177143,0
suma,1048576,8,R=47,0.000690674,0.000090701,0
suma,1048576,16,R=47,0.000690674,0.000058723,0
suma,2097152,1,R=23,0.001372658,0.001389602,0
suma,2097152,2,R=23,0.001372658,0.000726645,0
suma,2097152,4,R=23,0.001372658,0.000371933,0
suma,2097152,8,R=23,0.001372658,0.000192420,0
```

```
suma,2097152,16,R=23,0.001372658,0.000101062,0
suma,4194304,1,R=11,0.002709234,0.002766030,0
suma,4194304,2,R=11,0.002709234,0.001653276,0
suma,4194304,4,R=11,0.002709234,0.000766923,0
suma,4194304,8,R=11,0.002709234,0.000518018,0
suma,4194304,16,R=11,0.002709234,0.000378989,0

$ ./lab12 suma > suma_reto_run3.csv
[exit 0]
(checksum 5.576e+05)
$ cat suma_reto_run3.csv
modo,N,p,param,t_seq,t_par,t_parte_seq
suma,65536,1,R=762,0.000042970,0.000040872,0
suma,65536,2,R=762,0.000042970,0.000022239,0
suma,65536,4,R=762,0.000042970,0.000011750,0
suma,65536,8,R=762,0.000042970,0.000006757,0
suma,65536,16,R=762,0.000042970,0.000004665,0
suma,131072,1,R=381,0.000086916,0.000086711,0
suma,131072,2,R=381,0.000086916,0.000044291,0
suma,131072,4,R=381,0.000086916,0.000022169,0
suma,131072,8,R=381,0.000086916,0.000012017,0
suma,131072,16,R=381,0.000086916,0.000008065,0
suma,262144,1,R=190,0.000171457,0.000168425,0
suma,262144,2,R=190,0.000171457,0.000088355,0
suma,262144,4,R=190,0.000171457,0.000046293,0
suma,262144,8,R=190,0.000171457,0.000024393,0
suma,262144,16,R=190,0.000171457,0.000019898,0
suma,524288,1,R=95,0.000341072,0.000338506,0
suma,524288,2,R=95,0.000341072,0.000174855,0
suma,524288,4,R=95,0.000341072,0.000089758,0
suma,524288,8,R=95,0.000341072,0.000050698,0
suma,524288,16,R=95,0.000341072,0.000025700,0
suma,1048576,1,R=47,0.000686788,0.000685133,0
suma,1048576,2,R=47,0.000686788,0.000346691,0
suma,1048576,4,R=47,0.000686788,0.000172915,0
suma,1048576,8,R=47,0.000686788,0.000093077,0
suma,1048576,16,R=47,0.000686788,0.000058281,0
suma,2097152,1,R=23,0.001370200,0.001397283,0
suma,2097152,2,R=23,0.001370200,0.000705258,0
suma,2097152,4,R=23,0.001370200,0.000379160,0
suma,2097152,8,R=23,0.001370200,0.000220571,0
suma,2097152,16,R=23,0.001370200,0.000291968,0
suma,4194304,1,R=11,0.002823853,0.002831978,0
suma,4194304,2,R=11,0.002823853,0.001573646,0
suma,4194304,4,R=11,0.002823853,0.000795521,0
suma,4194304,8,R=11,0.002823853,0.000414066,0
suma,4194304,16,R=11,0.002823853,0.000580511,0

$ ./lab12 suma > suma_reto_run4.csv
[exit 0]
(checksum 5.576e+05)
$ cat suma_reto_run4.csv
modo,N,p,param,t_seq,t_par,t_parte_seq
suma,65536,1,R=762,0.000043299,0.000042785,0
suma,65536,2,R=762,0.000043299,0.000022070,0
suma,65536,4,R=762,0.000043299,0.000011213,0
suma,65536,8,R=762,0.000043299,0.000006794,0
suma,65536,16,R=762,0.000043299,0.000116328,0
suma,131072,1,R=381,0.000087405,0.000083597,0
suma,131072,2,R=381,0.000087405,0.000044729,0
suma,131072,4,R=381,0.000087405,0.000024310,0
suma,131072,8,R=381,0.000087405,0.000012409,0
suma,131072,16,R=381,0.000087405,0.000122625,0
suma,262144,1,R=190,0.000173734,0.000168931,0
suma,262144,2,R=190,0.000173734,0.000093318,0
suma,262144,4,R=190,0.000173734,0.000045653,0
suma,262144,8,R=190,0.000173734,0.000024353,0
suma,262144,16,R=190,0.000173734,0.000131212,0
```

```
suma,524288,1,R=95,0.000383130,0.000354650,0
suma,524288,2,R=95,0.000383130,0.000178672,0
suma,524288,4,R=95,0.000383130,0.000096111,0
suma,524288,8,R=95,0.000383130,0.000062980,0
suma,524288,16,R=95,0.000383130,0.000142522,0
suma,1048576,1,R=47,0.000765488,0.000730528,0
suma,1048576,2,R=47,0.000765488,0.000363268,0
suma,1048576,4,R=47,0.000765488,0.000172690,0
suma,1048576,8,R=47,0.000765488,0.000098666,0
suma,1048576,16,R=47,0.000765488,0.000204373,0
suma,2097152,1,R=23,0.001408183,0.001393620,0
suma,2097152,2,R=23,0.001408183,0.000730650,0
suma,2097152,4,R=23,0.001408183,0.000354054,0
suma,2097152,8,R=23,0.001408183,0.000185298,0
suma,2097152,16,R=23,0.001408183,0.000359368,0
suma,4194304,1,R=11,0.002835487,0.002864130,0
suma,4194304,2,R=11,0.002835487,0.001631654,0
suma,4194304,4,R=11,0.002835487,0.000711686,0
suma,4194304,8,R=11,0.002835487,0.000439121,0
suma,4194304,16,R=11,0.002835487,0.000652790,0

$ ./lab12 suma > suma_reto_run5.csv
[exit 0]
(checksum 5.576e+05)
$ cat suma_reto_run5.csv
modo,N,p,param,t_seq,t_par,t_parte_seq
suma,65536,1,R=762,0.000044532,0.000043614,0
suma,65536,2,R=762,0.000044532,0.000022277,0
suma,65536,4,R=762,0.000044532,0.000012064,0
suma,65536,8,R=762,0.000044532,0.000006822,0
suma,65536,16,R=762,0.000044532,0.000115982,0
suma,131072,1,R=381,0.000084125,0.000085424,0
suma,131072,2,R=381,0.000084125,0.000043711,0
suma,131072,4,R=381,0.000084125,0.000022021,0
suma,131072,8,R=381,0.000084125,0.000011881,0
suma,131072,16,R=381,0.000084125,0.000117923,0
suma,262144,1,R=190,0.000173798,0.000175926,0
suma,262144,2,R=190,0.000173798,0.000086760,0
suma,262144,4,R=190,0.000173798,0.000046820,0
suma,262144,8,R=190,0.000173798,0.000024152,0
suma,262144,16,R=190,0.000173798,0.000134684,0
suma,524288,1,R=95,0.000355207,0.000329182,0
suma,524288,2,R=95,0.000355207,0.000170143,0
suma,524288,4,R=95,0.000355207,0.000090624,0
suma,524288,8,R=95,0.000355207,0.000046753,0
suma,524288,16,R=95,0.000355207,0.000134293,0
suma,1048576,1,R=47,0.000717710,0.000697183,0
suma,1048576,2,R=47,0.000717710,0.000374990,0
suma,1048576,4,R=47,0.000717710,0.000174709,0
suma,1048576,8,R=47,0.000717710,0.000095218,0
suma,1048576,16,R=47,0.000717710,0.000198276,0
suma,2097152,1,R=23,0.001412841,0.001389556,0
suma,2097152,2,R=23,0.001412841,0.000702055,0
suma,2097152,4,R=23,0.001412841,0.000352898,0
suma,2097152,8,R=23,0.001412841,0.000198215,0
suma,2097152,16,R=23,0.001412841,0.000283785,0
suma,4194304,1,R=11,0.002843023,0.002864648,0
suma,4194304,2,R=11,0.002843023,0.001490654,0
suma,4194304,4,R=11,0.002843023,0.000726145,0
suma,4194304,8,R=11,0.002843023,0.000516482,0
suma,4194304,16,R=11,0.002843023,0.000584854,0

# fin: 2026-10-10 18:30:27
```

analizar_reto.py: resumen de las 5 corridas

archivo: evidencia/analisis_reto.txt

```
$ python3 analizar_reto.py resultados/reto
corridas: 5

N | MiB | p | S mediana | S min | S max | S>p en todas | S>p en alguna
65536 | 0.5 | 1 | 1.012 | 0.985 | 1.051 | no | 3/5
65536 | 0.5 | 2 | 1.932 | 1.888 | 1.999 | no | 0/5
65536 | 0.5 | 4 | 3.657 | 3.542 | 3.862 | no | 0/5
65536 | 0.5 | 8 | 6.373 | 5.266 | 6.528 | no | 0/5
65536 | 0.5 | 16 | 7.020 | 0.372 | 9.211 | no | 0/5
131072 | 1 | 1 | 1.002 | 0.985 | 1.046 | no | 3/5
131072 | 1 | 2 | 1.962 | 1.925 | 2.003 | no | 1/5
131072 | 1 | 4 | 3.819 | 3.595 | 3.921 | no | 0/5
131072 | 1 | 8 | 7.081 | 6.277 | 7.238 | no | 0/5
131072 | 1 | 16 | 10.694 | 0.713 | 11.275 | no | 0/5
262144 | 2 | 1 | 1.001 | 0.965 | 1.028 | no | 3/5
262144 | 2 | 2 | 1.941 | 1.827 | 2.003 | no | 1/5
262144 | 2 | 4 | 3.712 | 3.579 | 3.806 | no | 0/5
262144 | 2 | 8 | 7.134 | 6.961 | 7.236 | no | 0/5
262144 | 2 | 16 | 8.617 | 1.290 | 11.856 | no | 0/5
524288 | 4 | 1 | 1.008 | 0.962 | 1.080 | no | 4/5
524288 | 4 | 2 | 1.951 | 1.841 | 2.144 | no | 2/5
524288 | 4 | 4 | 3.840 | 3.800 | 3.986 | no | 0/5
524288 | 4 | 8 | 6.998 | 6.083 | 7.598 | no | 0/5
524288 | 4 | 16 | 2.688 | 2.319 | 13.271 | no | 0/5
1048576 | 8 | 1 | 1.029 | 1.002 | 1.048 | SI | 5/5
1048576 | 8 | 2 | 1.981 | 1.914 | 2.107 | no | 2/5
1048576 | 8 | 4 | 3.972 | 3.689 | 4.433 | no | 2/5
1048576 | 8 | 8 | 7.539 | 7.379 | 7.758 | no | 0/5
1048576 | 8 | 16 | 11.762 | 3.620 | 14.029 | no | 0/5
2097152 | 16 | 1 | 0.988 | 0.981 | 1.017 | no | 2/5
2097152 | 16 | 2 | 1.943 | 1.889 | 2.012 | no | 1/5
2097152 | 16 | 4 | 3.928 | 3.614 | 4.004 | no | 1/5
2097152 | 16 | 8 | 7.128 | 6.212 | 7.600 | no | 0/5
2097152 | 16 | 16 | 4.979 | 3.918 | 13.582 | no | 0/5
4194304 | 32 | 1 | 0.992 | 0.979 | 0.997 | no | 0/5
4194304 | 32 | 2 | 1.738 | 1.639 | 1.907 | no | 0/5
4194304 | 32 | 4 | 3.643 | 3.533 | 3.984 | no | 0/5
4194304 | 32 | 8 | 6.228 | 5.230 | 6.820 | no | 0/5
4194304 | 32 | 16 | 4.864 | 4.344 | 7.149 | no | 0/5

costo secuencial por elemento (ns), mediana de las corridas:
  N=65536 (0.5 MiB): 0.656 ns  (min 0.640, max 0.680)
  N=131072 (1 MiB): 0.663 ns  (min 0.642, max 0.667)
  N=262144 (2 MiB): 0.657 ns  (min 0.652, max 0.663)
  N=524288 (4 MiB): 0.655 ns  (min 0.647, max 0.731)
  N=1048576 (8 MiB): 0.668 ns  (min 0.655, max 0.730)
  N=2097152 (16 MiB): 0.656 ns  (min 0.653, max 0.674)
  N=4194304 (32 MiB): 0.673 ns  (min 0.646, max 0.678)
[exit 0]
```